CS 344: Design and Analysis of Computer Algorithms **Rutgers: Spring 2026**

Midterm Exam

Name: *Netid:*

Instructions

1. Write your name and NetID at the top of the exam.
2. The exam consists of 4 problems worth 25 points in total, plus one extra credit problem worth 20 points.
3. This is an open-notes exam. You may consult any course materials (e.g., lecture notes, textbook etc), but no electronic resources are allowed.
4. **Provide proofs of correctness for your algorithms and analyze their runtime**.
5. **Avoid overly complicated pseudo-code; explain your algorithm clearly in English**.
6. You may use any algorithm presented in class or homework as a building block for your solutions.

Problem #	**Points**	**Score**
1	25	
2	25	
3	25	
4	25	
5	+20	
Total	**100+20**	

Problem 1

a) For each of the following functions determine the strongest asymptotic relation between the functions i.e., whether $f(n) = o(g(n)), f(n) = O(g(n)), f(n) = \Omega(g(n)), f(n) = \omega(g(n))$, or $f(n) = \Theta(g(n))$. Prove the correctness of your choice.

1. $f(n) = n^2 \log n$ and $g(n) = e^{3 \log n}$
2. $f(n) = \log(n!) + n$ and $g(n) = 2n \log n$
3. $f(n) = \log(2n + 3)$ and $g(n) = f(\sqrt{n})$
4. $f(n) = \frac{n}{\log n}$ and $g(n) = \frac{n \log n}{e^{\log \log n + 2}}$

Solution:

1. $f(n) = n^2 \log n$ and $g(n) = e^{3 \log n}$
   Let's simplify $g(n)$:
   $$\begin{aligned} g(n) &= e^{3 \log n} \\ &= e^{\log n^3} \\ &= n^3. \end{aligned}$$
   We know that $\log n = o(n)$, therefore
   $$n^2 \log n = o(n^3).$$
   So we have $f(n) = o(g(n))$.
2. $f(n) = \log(n!) + n$ and $g(n) = 2n \log n$
   We use that $\log n! = \theta(n \log n)$, so we can write:
   $$\begin{aligned} f(n) &= \log(n!) + n \\ &= \theta(n \log n) + n \\ &= \theta(n \log n). \end{aligned}$$
   Since both $f(n)$ and $g(n)$ are $\theta(n \log n)$, we have:
   $$f(n) = \theta(g(n)).$$
3. $f(n) = \log(2n + 3)$ and $g(n) = f(\sqrt{n})$
   We simplify $f(n)$:
   $$\begin{aligned} f(n) &= \log(2n + 3) \\ &= \theta(\log(2n)) \\ &= \theta(\log n). \end{aligned}$$
   Now simplify $g(n)$:
   $$\begin{aligned} g(n) &= f(\sqrt{n}) \\ &= \log(2\sqrt{n} + 3) \\ &= \theta(\log(2\sqrt{n})) \\ &= \theta(\log \sqrt{n}) \\ &= \theta\left(\frac{1}{2} \log n\right) \\ &= \theta(\log n). \end{aligned}$$

Thus, we have:

$$f(n) = \theta(g(n)) = \theta(\log n).$$

4. $f(n) = \frac{n}{\log n}$ and $g(n) = \frac{n \log n}{e^{\log \log n + 2}}$

   Let's simplify $g(n)$:

$$\begin{aligned} g(n) &= \frac{n \log n}{e^{\log \log n + 2}} \\ &= \frac{n \log n}{\log n \cdot e^2} \\ &= \frac{n}{e^2}. \end{aligned}$$

   Since $f(n) = \frac{n}{\log n} = o(n)$, we conclude that:

$$f(n) = o(g(n)).$$

b) Consider two large $3n \times 3n$ matrices, A and B, which are divided into nine $n \times n$ submatrices:

$$A = \begin{pmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} & B_{13} \\ B_{21} & B_{22} & B_{23} \\ B_{31} & B_{32} & B_{33} \end{pmatrix}$$

When computing $C = A \times B$, the naive approach requires 27 scalar multiplications for the 3×3 block matrix multiplication. However, optimizations such as Laderman's algorithm reduce this to 23 multiplications,

1. For each of the following approaches, write down the recurrence relation for the time complexity $T(n)$ when applied recursively to $3n \times 3n$ matrices:
   - **Naive Approach:** Using 27 multiplications.
   - **Laderman's Algorithm:** Using 23 multiplications.
2. Solve each recurrence relation using a recursion tree to find the asymptotic time complexity for each method.

Solution:

1. The naive approach will use 27 multiplications of size $n/3$ and a constant number of additions. Each addition of the matrices takes $\Theta(n^2)$ time. Thus, we have the recurrence relation:

   $$T(n) = 27T\left(\frac{n}{3}\right) + \Theta(n^2).$$

   At each level, the problem size is divided by 3 as we go down the recursion tree, so there are $\log_3 n$ levels in the tree (the depth is $\log_3 n$).

   At level i, there are 27^i subproblems, each of size $\frac{n}{3^i}$, and each such subproblem incurs a cost of $C \cdot \left(\frac{n}{3^i}\right)^2$ while combining solutions to its subproblems. Hence, by summing up all the values, we get:

   $$T(n) = \sum_{i=0}^{k} 27^i \cdot C \cdot \left(\frac{n}{3^i}\right)^2 = C \cdot n^2 \cdot \sum_{i=0}^{k=\log_3 n} \left(\frac{27}{9}\right)^i = C \cdot n^2 \cdot \sum_{i=0}^{k=\log_3 n} 3^i.$$

   This is a geometric series that grows exponentially and is dominated by the last term, so we have:

   $$T(n) = \Theta(n^2 \cdot 3^k).$$

   Since $n = 3^k$, we have:

   $$T(n) = \Theta(n^2 \cdot n) = \Theta(n^3).$$

2. Now, consider Laderman's approach, which takes 23 multiplications and a constant number of additions. The recurrence becomes:

   $$T(n) = 23T\left(\frac{n}{3}\right) + \Theta(n^2).$$

   Once again, at each level, the problem size is divided by 3, so there are $\log_3 n$ levels in the recursion tree.

   At level i, there are 23^i subproblems, each of size $\frac{n}{3^i}$, and each incurs a cost of $C \cdot \left(\frac{n}{3^i}\right)^2$ for combining the subproblem solutions. Summing over all levels, we get:

$$T(n) = \sum_{i=0}^{k} 23^i \cdot C \cdot \left(\frac{n}{3^i}\right)^2 = C \cdot n^2 \cdot \sum_{i=0}^{k=\log_3 n} \left(\frac{23}{9}\right)^i .$$

This is again a geometric series dominated by the last term, so we have:

$$T(n) = \Theta\left(n^2 \cdot \left(\frac{23}{9}\right)^k\right).$$

Since $n = 3^k$, we can express this as:

$$\left(\frac{23}{9}\right)^k = 3^{k \cdot \log_3\left(\frac{23}{9}\right)} = n^{\log_3\left(\frac{23}{9}\right)}.$$

Thus, the final result is:

$$T(n) = \Theta\left(n^2 \cdot n^{\log_3\left(\frac{23}{9}\right)}\right) = \Theta\left(n^{\log_3 23}\right).$$

Problem 2

In this problem, we consider a non-standard sorting algorithm called the *Magic Sort*. Given an array $A[1:n]$ of n integers, the algorithm is as follows:

Magic-Sort$(A[1:k])$:

1. If $k < \frac{n}{100}$, run merge sort on A.
2. Otherwise, run **Magic-Sort**$(A[1:n-1])$, **Magic-Sort**$(A[2:n])$, and **Magic-Sort**$(A[1:n-1])$ again.

We now analyze this algorithm.

(a) Prove the correctness of **Magic-Sort**.

(b) Write a recurrence for **Magic-Sort** and solve this recurrence and find the *tightest asymptotic* upper bound on the runtime.

solution:

We will prove this via induction.

Our base case is when $k < \frac{n}{100}$. The induction hypothesis is true in this case because we know that merge sort is correct, so Magic-Sort must be correct as all it does is call merge sort for smaller arrays (size $n/100$ or less) and return the same answer.

Now, we prove the induction step: assume that **Magic-Sort** is correct for all arrays of length $n \leq l$, and we will show that it is also correct for arrays of length $n = l + 1$.

Note that since the size of the inner instances passed to **Magic-Sort** is strictly less than n, we can assume by the induction hypothesis that these smaller subarrays are being sorted correctly (locally). We now analyze what happens at each step:

- After the first run of **Magic-Sort**$(A[1:n-1])$, the largest number in the entire array A will be in either position $A[n-1]$ or $A[n]$. This is because we correctly sorted $A[1:n-1]$ locally, so if the largest element of the array was in the range $[1:n-1]$, it will now be placed in $A[n-1]$. Otherwise, if the maximum is already in $A[n]$, it remains there.
- After the next run of **Magic-Sort**$(A[2:n])$, the largest number in the entire array A will be in position $A[n]$. This happens because, from the previous part, we know the largest number is in the range $[2:n]$ that we are sorting. Since **Magic-Sort** will sort $A[2:n]$ correctly (by induction), it will place the largest number at $A[n]$.
- After the final run of **Magic-Sort**$(A[1:n-1])$, all elements of A will be sorted correctly. The largest number in the array is already in $A[n]$, and all elements of $A[1:n-1]$ are sorted (by the previous part and the induction hypothesis). Therefore, after sorting $A[1:n-1]$, A will be fully sorted.

The above proves that A is now sorted, hence proving the induction step. This completes the proof of correctness of the algorithm by induction.

The recurrence for **Magic-Sort** is given by:

$$T(n) = 3 \cdot T(n-1) + O(1),$$

because we recursively call three instances of size $n-1$. The $O(1)$ term accounts for the time spent making the recursive calls and performing other constant-time operations, but it doesn't significantly affect the overall complexity (as shown in the HW1 solution).

By using either the recurrence tree method or direct substitution, we observe that:

$$T(n) = \Theta(3^k \cdot T(n-k)).$$

We continue expanding the recurrence until the problem size reduces to $n/100$, i.e., until $n-k = n/100$, which gives us $k = 99n/100$. Substituting this into the recurrence, we get:

$$T(n) = \Theta\left(3^{99n/100} \cdot T(n/100)\right).$$

At this point, the algorithm switches to merge sort for the base case, which has a complexity of $\Theta(n \log n)$ for arrays of size $n/100$. Thus, we have:

$$T(n) = \Theta\left(3^{99n/100} \cdot n \log n\right).$$

Therefore, the overall complexity of **Magic-Sort** is dominated by the exponential term $3^{99n/100}$, making the final time complexity:

$$T(n) = \Theta\left(3^{99n/100} n \log n\right).$$

Problem 3

Write an algorithm for the following:

- Input: An array A with n distinct (non-equal) elements
- Output-1: a pair of numbers w and z in A that minimize $|w + z + 2|$.
  The run-time should be significantly better than $O(n^2)$.
- Output-2: a pair of numbers x and y such that $2x + y = 0$ The expected runtime should be $O(n)$.

Solution. Output-1: Minimizing $|w + z + 2|$

This is basically 2-sum problem, but we are trying to find a sum closest to a target (here -2) instead of exactly equal to target. So as we do the two pointer approach we need to keep track of a closest solution we get and update it if we find a better one.

Algorithm:

1. **Sort the array:** Sort the array A in ascending order. Let the sorted array be A_{sorted}.
2. **Initialize:**
   - Set a pointer $i = 1$ at the beginning of the array and a pointer $j = n$ at the end.
   - Let min_val $= \infty$ be a variable to track the smallest distance found so far.
   - Let best_pair be a variable to store the resulting numbers w and z.
3. **Two-Pointer Search:** While $i < j$:
   (a) Calculate the current sum offset: $s = A_{\text{sorted}}[i] + A_{\text{sorted}}[j] + 2$.
   (b) If $|s| <$ min_val:
      - Update min_val $= |s|$.
      - Set best_pair $= (A_{\text{sorted}}[i], A_{\text{sorted}}[j])$.
   (c) If $s > 0$: Decrease j by 1 (to make the sum smaller).
   (d) Else if $s < 0$: Increase i by 1 (to make the sum larger).
   (e) Else ($s = 0$): We found an exact match. Return the current pair immediately.
4. **Output:** Return the pair stored in best_pair.

Runtime Analysis: Sorting the array takes $O(n \log n)$ time. The two-pointer walk takes $O(n)$ time as each step moves one of the pointers closer to the other. The total runtime is $O(n \log n)$, which is significantly better than $O(n^2)$.

Output-2: Finding a Pair for $2x + y = 0$ in $O(n)$ expected time.

Algorithm:

1. **Initialize:** Create an empty randomized hash table H.
2. **Single-Pass Search:** For each element v in array A:
   (a) **Check if v is y:** See if the value $x = -v/2$ exists in the hash table H. If it does, return the pair $(-v/2, v)$.
   (b) **Check if v is x:** See if the value $y = -2v$ exists in the hash table H. If it does, return the pair $(v, -2v)$.

(c) **Insert:** If no match is found, insert v into the hash table H and continue to the next element.

3. **Output:** If the loop completes without a match, return that no such pair exists.

Runtime Analysis: Creating and inserting elements into a randomized hash table takes $O(1)$ expected time per operation. Since we iterate through the n elements once and perform constant-time lookups and insertions, the total expected runtime is $O(n)$.

Problem 4: Finding the Maximum in a Bimodal Array

Problem: We are given a bimodal array A of n distinct elements. The sequence follows a specific pattern: it increases, then decreases, then increases again, and finally decreases. We are given that the local minimum of this "valley" occurs exactly at the middle index $n/2$. Our goal is to find the global maximum element in $O(\log n)$ time.

Algorithm: The core idea is to treat the bimodal array as two separate unimodal sub-problems. A unimodal array (which increases then decreases) can be searched for its peak in logarithmic time using a modified binary search.

1. **Divide the Array:** Split the array A into two halves based on the known local minimum at $mid = n/2$:
   - $L = A[1 \dots n/2]$
   - $R = A[n/2 + 1 \dots n]$

   By the problem definition, both L and R are now unimodal (they each contain exactly one peak).

2. **Binary Search for Peaks:** Define a procedure $\textsc{FindPeak}(S)$ that finds the maximum of a unimodal array S in $O(\log |S|)$ time:
   (a) Let $low = 1$ and $high = |S|$.
   (b) While $low < high$:
      - Pick a midpoint $m = \lfloor (low + high)/2 \rfloor$.
      - Compare $S[m]$ with its neighbor $S[m + 1]$.
      - If $S[m] < S[m + 1]$, we are in the increasing portion; set $low = m + 1$.
      - If $S[m] > S[m + 1]$, we are in the decreasing portion; set $high = m$.

   (c) Return $S[low]$.

3. **Global Comparison:** Apply the peak-finding procedure to both halves:

$$m_1 = \textsc{FindPeak}(L), \quad m_2 = \textsc{FindPeak}(R)$$

   The global maximum is $\max(m_1, m_2)$.

Proof of Correctness: A bimodal array with a minimum at the center is guaranteed to be the concatenation of two unimodal segments. In a unimodal segment, the property $A[i] < A[i+1]$ holds if and only if the peak is to the right of i. Conversely, $A[i] > A[i + 1]$ implies the peak is at i or to its left. This "binary" property allows the search space to be halved at each step, ensuring the local peak of each segment is found correctly. Since the global maximum must be one of these two local peaks, the algorithm is correct.

Time Complexity Analysis:

- Splitting the array takes $O(1)$ time (by manipulating indices).
- Each call to $\textsc{FindPeak}$ takes $O(\log(n/2)) = O(\log n)$ time because it is a standard binary search.
- Comparing the two results takes $O(1)$ time.

The total runtime is $O(\log n)$, which is significantly better than the $O(n)$ required for a linear scan.

Problem 5: Finding the K Closest Elements

Problem: Given an array A of n integers and a target value T, we want to find the K elements in A that have the smallest absolute difference from T.

Algorithm: To solve this using a Min-Heap, we will prioritize efficiency by building the heap from all distances and then extracting the K closest values.

1. **Calculate Distances:** For every element $A[i]$ in the array, compute its distance to the target: $d_i = |A[i] - T|$. Store these as pairs $(d_i, A[i])$.
2. **Build a Min-Heap:** Construct a Min-Heap H using all n distance pairs. We use the distance d_i as the key for the heap.
   - Using the bottom-up build-heap algorithm, this step can be completed in $O(n)$ time.
3. **Extract K Elements:** Perform the extract-min operation on the heap exactly K times.
   - Each extract-min returns the element with the smallest remaining distance to T.
   - Store each extracted element in a result list.
4. **Output:** Return the list of K extracted elements.

Correctness: A Min-Heap is designed to always provide the element with the minimum key at the root. By using the absolute distances as keys, the first K elements removed via extract-min are guaranteed to be the K elements with the smallest distances to the target T in the entire array.

Time Complexity Analysis:

- **Distance Calculation:** Takes $O(n)$ to iterate through the array once.
- **Building the Heap:** Constructing a Min-Heap from n elements takes $O(n)$ time using the linear-time build-heap procedure.
- **Extracting K Elements:** Each extract-min operation takes $O(\log n)$ time. Doing this K times takes $O(K \log n)$.
- **Total Complexity:** The total runtime is $O(n + K \log n)$.

Problem 6

You are given a set of n items with weights $w_1, \ldots, w_n$ and values $v_1, \ldots, v_n$. You are also given two knapsacks with sizes W_1, W_2. The goal is to pick a subset of items with maximum value such that we can place each of the chosen items in one of the two knapsacks with the restriction that the total weight of items in each knapsack should be smaller than its size. Design an $O(n \cdot W_1 \cdot W_2)$ time dynamic programming algorithm that outputs the maximum value we can obtain by picking a subset of items that fits the two knapsacks.

Remember to separately write your specification of recursive formula in plain English , your recursive solution for the formula and its proof of correctness , the algorithm using either memoization or bottom-up dynamic programming , and runtime analysis.

Solution:

Specification: For every $0 \leq i \leq n$, $0 \leq j \leq W_1$, and $0 \leq k \leq W_2$, we define:

- $K(i, j, k)$: the maximum value we can obtain by picking a subset of items $\{1, \ldots, i\}$ that fits in a knapsack of size j and another knapsack of size k.

The solution to our original problem is then $K(n, W_1, W_2)$.

Recursive formula: We design the following recursive formula:

$$K(i,j,k) = \begin{cases} 0, & \text{if } i = 0 \text{ or } j = k = 0 \\ K(i-1,j,k), & \text{if } w_i > j \text{ and } w_i > k \\ \max\{K(i-1,j,k), K(i-1,j-w_i,k) + v_i\}, & \text{if } w_i \leq j \text{ and } w_i > k \\ \max\{K(i-1,j,k), K(i-1,j,k-w_i) + v_i\}, & \text{if } w_i > j \text{ and } w_i \leq k \\ \max\{K(i-1,j,k), K(i-1,j-w_i,k) + v_i, K(i-1,j,k-w_i) + v_i\}, & \text{otherwise.} \end{cases}$$

Correctness: We now prove the correctness of this algorithm:

- If $i = 0$, there is no item to pick, and if both $j = k = 0$, then there is no way to fit any item in either of the knapsacks. Hence, $K(i, j, k) = 0$ is the correct value.
- If $w_i > j$ and $w_i > k$, item i will not fit in either knapsack. Thus, the only thing we can do is skip this item, and from the items $\{1, \ldots, i-1\}$, pick the best solution (corresponding to $K(i-1, j, k)$). So, $K(i, j, k) = K(i-1, j, k)$ is the correct value.
- If $w_i \leq j$ but $w_i > k$, we have two options:
  1. Ignore item i, collecting the value $K(i-1, j, k)$ (as described in the previous case).
  2. Pick item i in the first knapsack, collecting value v_i. This reduces the capacity of knapsack one to $j - w_i$, and we then pick the best solution from the remaining items (corresponding to $K(i-1, j-w_i, k)$). The total value is $K(i-1, j-w_i, k) + v_i$.

  The goal is to maximize the value obtained, so the correct choice is:

  $$K(i,j,k) = \max\{K(i-1,j,k), K(i-1,j-w_i,k) + v_i\}.$$

  Note that item i cannot fit in the second knapsack, as $w_i > k$.
- If $w_i > j$ but $w_i \leq k$, we can proceed as in the previous case, but this time placing item i in the second knapsack instead of the first. Hence, the correct formula is:

  $$K(i,j,k) = \max\{K(i-1,j,k), K(i-1,j,k-w_i) + v_i\}.$$

- If $w_i \leq j$ and $w_i \leq k$, we have three options:
  1. Skip item i, collecting the value $K(i-1, j, k)$.
  2. Pick item i in the first knapsack, collecting $K(i-1, j-w_i, k) + v_i$.
  3. Pick item i in the second knapsack, collecting $K(i-1, j, k-w_i) + v_i$.

  The correct choice is to maximize these three options:

$$K(i, j, k) = \max\{K(i-1, j, k), K(i-1, j-w_i, k) + v_i, K(i-1, j, k-w_i) + v_i\}.$$

Bottom-Up Dynamic Programming

We give a bottom-up dynamic programming algorithm for the problem (you could also use memoization, which would yield the same result). We need to pick an evaluation order: since each $K(i, j, k)$ is updated from $K(i', j', k')$ where $i' < i, j' \leq j$, and $k' \leq k$, we just need to ensure that we iterate over all i, j, k in increasing order of their values. The algorithm proceeds as follows:

1. Let $T[0:n][0:W_1][0:W_2]$ be a 3D array of size $(n+1) \times (W_1+1) \times (W_2+1)$, initialized with 0s. This automatically handles the base case where no items are chosen.
2. For each $i = 1$ to n, do the following:
   (a) For each $j = 1$ to W_1, do the following:
      i. For each $k = 1$ to W_2, do the following:
         A. If $w_i > j$ and $w_i > k$, then set:
$$T[i][j][k] = T[i-1][j][k].$$
         B. Else if $w_i \leq j$ but $w_i > k$, then set:
$$T[i][j][k] = \max(T[i-1][j][k], T[i-1][j-w_i][k] + v_i).$$
         C. Else if $w_i > j$ but $w_i \leq k$, then set:
$$T[i][j][k] = \max(T[i-1][j][k], T[i-1][j][k-w_i] + v_i).$$
         D. Otherwise, set:
$$T[i][j][k] = \max(T[i-1][j][k], T[i-1][j-w_i][k] + v_i, T[i-1][j][k-w_i] + v_i).$$
3. Return $T[n][W_1][W_2]$, which contains the maximum value that can be obtained for the given weights W_1 and W_2.

Runtime analysis:

- The first line of the algorithm, initializing the array T, takes $O(n \cdot W_1 \cdot W_2)$ time.
- The algorithm consists of three nested loops: the outer loop runs for n iterations, the middle loop runs for W_1 iterations, and the innermost loop runs for W_2 iterations. Each iteration of the innermost loop takes $O(1)$ time. Therefore, the total runtime is:
$$O(n \cdot W_1 \cdot W_2).$$
- For a memoization algorithm, the runtime is the same: there are $O(n \cdot W_1 \cdot W_2)$ subproblems $K(i, j, k)$, and each subproblem takes $O(1)$ time to compute. Hence, the total runtime is again:
$$O(n \cdot W_1 \cdot W_2).$$

Problem 7: Counting Inversions using Divide and Conquer

Problem: Given an array A of n distinct elements, we want to count the number of pairs (i, j) such that $i < j$ but $A[i] > A[j]$. These pairs are called *inversions*.

Algorithm: The algorithm is based on a modified MergeSort. We recursively divide the array into halves, count the inversions within each half, and then count the "split inversions" (where one element is in the left half and the other is in the right) during the merge step.

1. **Base Case:** If the array has only one element, it contains 0 inversions. Return 0.
2. **Divide and Conquer:**
   - Divide the array into two halves: $L = A[1 \dots \lfloor n/2 \rfloor]$ and $R = A[\lfloor n/2 \rfloor + 1 \dots n]$.
   - Let $X = \text{CountInversions}(L)$.
   - Let $Y = \text{CountInversions}(R)$.
   - After these calls, L and R are now sorted.
3. **Merge and Count Split Inversions:** Create a new sorted array while counting how many elements from L are greater than elements from R. Initialize $Z = 0$, $i = 1$ (for L), and $j = 1$ (for R).
   - While $i \leq |L|$ and $j \leq |R|$:
     - If $L[i] < R[j]$: $L[i]$ is not in an inversion with $R[j]$ or any subsequent element in R. Move $L[i]$ to the output and $i \leftarrow i + 1$.
     - If $L[i] > R[j]$: Since L is sorted, $R[j]$ is smaller than $L[i]$ and *every* element following it in L. There are $(|L| - i + 1)$ such elements.
     - Increment $Z \leftarrow Z + (|L| - i + 1)$.
     - Move $R[j]$ to the output and $j \leftarrow j + 1$.
4. **Output:** Return the total sum $X + Y + Z$.

Correctness: Every inversion (i, j) falls into one of three categories: it is entirely in the left half, entirely in the right half, or split across both. The recursive calls correctly count the first two types. For split inversions, since L and R are sorted, if $L[i] > R[j]$, then $L[k] > R[j]$ for all $k > i$. The algorithm counts all such pairs exactly once as the elements of R are merged into the final array. Because the merge process considers every pair of elements where one is from L and one is from R, no split inversion is missed.

Time Complexity Analysis: The algorithm follows the same recurrence as MergeSort. Let $T(n)$ be the time to count inversions in an array of size n:

$$T(n) = 2T(n/2) + O(n)$$

- The $2T(n/2)$ term accounts for the two recursive calls.
- The $O(n)$ term accounts for the merge step and the arithmetic used to increment the inversion counter.

The recursion tree contributes $O(n)$ at each level and hence the total work for $O(\log n)$ levels gives $T(n) = O(n \log n)$.

Problem 8

Problem: We are given two arrays: g (greed factors), where $g[i]$ is the minimum cookie size required to satisfy child i, and s (cookie sizes), where $s[j]$ is the size of cookie j. Each child can receive at most one cookie, and each cookie can be given to at most one child. Our goal is to maximize the number of content children.

Algorithm: We employ a Greedy Strategy. The intuition is that to maximize the total number of satisfied children, we should not "waste" large cookies on children with small appetites. We should satisfy the child with the smallest appetite using the smallest possible cookie that meets their requirement.

1. Sort both arrays g and s in non-decreasing order.
2.
   - Initialize two pointers, $i = 1$ (for children) and $j = 1$ (for cookies).
   - Initialize a counter $count = 0$.
3. **Iteration:** While $i \leq |g|$ and $j \leq |s|$:
   (a) If $s[j] \geq g[i]$:
      - This cookie satisfies the child. Increment $count \leftarrow count + 1$.
      - Move to the next child: $i \leftarrow i + 1$.
   (b) Regardless of whether the child was satisfied, move to the next cookie: $j \leftarrow j + 1$.
   (c) *Note:* If $s[j] < g[i]$, the current cookie is too small for the current child (and any child with a larger greed factor), so we discard it.
4. **Output:** Return *count*.

Proof of Correctness (Exchange Argument)

We prove that the Greedy Algorithm is optimal by showing that any optimal solution O can be transformed into the greedy solution G without reducing the number of satisfied children.

Proof: Let $G = \{(c_{i_1}, s_{j_1}), (c_{i_2}, s_{j_2}), \ldots, (c_{i_k}, s_{j_k})\}$ be the set of pairings produced by our greedy algorithm, and let O be an arbitrary optimal matching. We will transform O into G step-by-step.

Suppose the first m choices of G match O. Consider the $(m+1)$-th greedy choice, where the greedy algorithm matches the smallest remaining child c with the smallest possible satisfying cookie s. There are three cases for what O does with child c:

1. **Child c is matched with the same cookie s in O:** In this case, the $(m + 1)$-th choice already matches, and we move to the next choice.
2. **Child c is matched with a different cookie s' in O:** By the greedy property, s is the smallest cookie that satisfies c, so $s \leq s'$.
   - If s was unmatched in O: We can simply replace the pair (c, s') with (c, s) in O. Since $s \leq s'$, child c is still satisfied. O now matches G for this child, and the number of satisfied children remains the same.
   - If s was matched with another child c' in O: We swap the cookies. O originally had (c, s') and (c', s). We change these to (c, s) and (c', s'). Since $s' \geq s$ and c' was satisfied by s ($s \geq g[c']$), it follows that $s' \geq g[c']$. Thus, c' is still satisfied by s'. The total number of satisfied children does not decrease.
3. **Child c is not matched at all in O:**

- If the greedy cookie s is also unmatched in O, we can simply add the pair (c, s) to O, which would mean O was not actually optimal (as we increased the count).
- If the greedy cookie s is matched with some other child c' in O, we replace (c', s) with (c, s). Since s satisfies c and c has the smallest greed factor among remaining children, this exchange is valid. We satisfy c instead of c', maintaining the same total count.

By repeating this exchange process for every greedy choice, we transform O into G without ever decreasing the number of satisfied children. Thus, the greedy algorithm produces an optimal solution. $\square$

Time Complexity Analysis:

- **Sorting:** Sorting g takes $O(n \log n)$ and sorting s takes $O(m \log m)$, where n and m are the lengths of the arrays.
- **Matching:** The two-pointer approach involves a single pass through both arrays, taking $O(n + m)$ time.
- **Total Runtime:** $O(n \log n + m \log m)$. This is dominated by the sorting step.